

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 04 -

ARQUITETURAS DE SERVIÇOS E CLEAN ARCHITECTURE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	18

EMSE

O QUE VEM POR AÍ?

Modelos de Machine Learning só geram valor quando fazem parte de sistemas bem arquitetados. Aqui, o foco sai do notebook e entra na realidade de aplicações de ML em produção: são exploradas decisões arquiteturais que impactam escalabilidade, testes e manutenção. A comparação entre monólitos e microsserviços mostra quando cada abordagem faz sentido e Clean Architecture e Arquitetura Hexagonal revelam como proteger regras de ML do acoplamento técnico.

O domínio passa a ocupar o centro do sistema e a infraestrutura se torna substituível. Conceitos introdutórios de Domain-Driven Design organizam código a partir do negócio e experimentos, métricas e validações ganham significado arquitetural. Padrões como Repository e Unit of Work entram como visão de sistemas maduros. O resultado é Machine Learning tratado como software profissional, não como experimento.

HANDS ON

A etapa prática foi pensada para ir além do treino de modelos isolados e mostrar, na prática, como Machine Learning se encaixa em sistemas reais. O objetivo não é apenas executar algoritmos, mas estruturar código, organizar responsabilidades e aplicar padrões arquiteturais amplamente utilizados em projetos profissionais de ML.

Durante o hands on, o(a) estudante vai construir gradualmente uma aplicação de Machine Learning com foco em clareza, separação de responsabilidades e evolutividade. Cada decisão será guiada por problemas comuns enfrentados em ambientes reais, como acoplamento excessivo, dificuldade de testes, dependência de tecnologia e falta de governança sobre modelos treinados.

O ponto de partida será a organização do projeto, saindo de uma abordagem típica de notebook para uma estrutura de código orientada a domínio. A partir disso, serão introduzidos conceitos de Clean Architecture e Arquitetura Hexagonal, mostrando como manter a lógica de Machine Learning — como regras de treino, validações e métricas — isolada das camadas de infraestrutura — como banco de dados, arquivos ou APIs externas.

Em seguida, o hands on apresenta uma visão prática de Domain-Driven Design (DDD), modelando conceitos centrais como experimentos de ML, métricas e hiperparâmetros de forma explícita no código.

A persistência de dados e modelos será abordada de forma arquitetural, utilizando o padrão Repository para abstrair o acesso aos dados de treinamento e resultados. Isso permitirá entender como a mesma lógica de ML pode operar sobre diferentes fontes de dados sem alterações no núcleo do sistema. O Unit of Work será apresentado de forma conceitual, conectando a ideia de transações ao processo de treino e validação de modelos, preparando você para desafios mais avançados.

Ao final, o hands on consolida uma visão essencial para quem deseja atuar profissionalmente com Machine Learning: modelos são apenas uma parte da solução. O verdadeiro diferencial está em como o ML é integrado ao software, garantindo qualidade, reprodutibilidade, manutenção e crescimento do sistema ao longo do tempo.

SAIBA MAIS

Projetos de Machine Learning raramente falham por falta de algoritmos sofisticados. Na prática, a maioria dos problemas surge quando uma solução que funciona bem em um experimento precisa evoluir para um sistema real, mantido por equipes, integrado a outras aplicações e executado continuamente em produção. É nesse ponto que a arquitetura de software se torna decisiva.

Em um primeiro momento, é comum que o trabalho em ML comece em um notebook experimental. Nesse contexto, o foco está em explorar dados, testar hipóteses, comparar modelos e avaliar métricas. O código costuma ser linear, fortemente acoplado aos dados disponíveis e orientado à experimentação rápida. Esse formato é adequado para pesquisa e prototipação, mas não foi pensado para manutenção, testes ou escalabilidade.

Um sistema de Machine Learning em produção, por outro lado, possui exigências muito diferentes. Ele precisa receber dados de fontes variadas, aplicar regras de negócio, treinar ou carregar modelos, gerar previsões de forma confiável e se integrar a outros serviços. Além disso, esse sistema deve ser testável, observável, versionável e preparado para mudanças frequentes — seja na fonte de dados, no modelo ou nas regras do domínio.

Quando não há uma arquitetura bem definida, alguns problemas tendem a aparecer rapidamente. O código se torna excessivamente acoplado, misturando lógica de negócio, acesso a dados e detalhes de infraestrutura. Isso dificulta a escrita de testes automatizados, torna a depuração mais complexa e aumenta o risco de erros em alterações simples. A escalabilidade também se torna limitada, pois pequenas mudanças exigem grandes retrabalhos. Trocar uma fonte de dados, substituir um algoritmo ou expor o modelo como serviço pode demandar modificações em várias partes do sistema.

Falar de arquitetura em Machine Learning significa, portanto, conectar modelos estatísticos e algoritmos à engenharia de software em larga escala. O objetivo não é apenas fazer previsões corretas, mas construir sistemas que evoluam ao longo do tempo de forma segura, organizada e sustentável.

Ao longo desta aula, serão apresentados padrões arquiteturais que ajudam a resolver esses desafios, como a separação clara de responsabilidades, o isolamento da lógica de negócio e a redução do acoplamento entre componentes. Esses conceitos permitem que soluções de ML sejam mais fáceis de testar, manter e escalar, independentemente das tecnologias utilizadas.

Como referência conceitual para essa abordagem, será utilizado o livro *Architecture Patterns with Python* (2020), que apresenta padrões modernos de arquitetura aplicáveis a sistemas reais, incluindo cenários onde dados, regras de negócio e infraestrutura precisam conviver de forma organizada.

Arquitetura de sistemas de machine learning

Um sistema de Machine Learning em produção não é composto apenas por um modelo treinado. Na prática, ele é formado por um conjunto de etapas interligadas, organizadas em um pipeline, que vai desde a obtenção dos dados até a entrega de previsões para outros sistemas. Entender esse pipeline é fundamental para projetar arquiteturas robustas, escaláveis e fáceis de manter.

Em arquiteturas bem definidas, cada etapa do pipeline possui uma responsabilidade clara, permitindo evolução independente, testes isolados e substituição de componentes sem impacto no sistema como um todo.

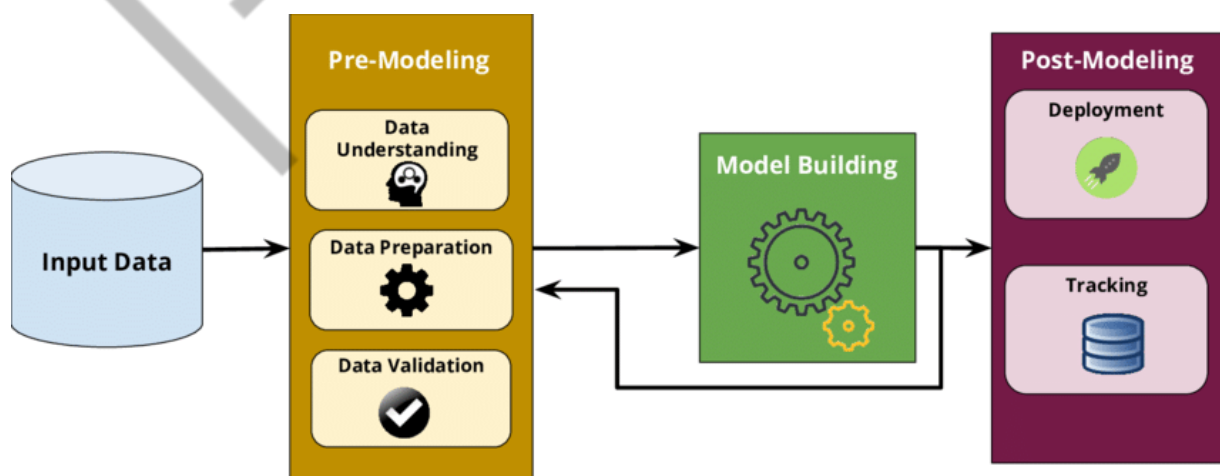


Figura 1 - Pipeline típico de ML em produção
Fonte: Sufyan (2022)

A ingestão de dados é o ponto de entrada do pipeline. Nessa etapa, o sistema coleta informações a partir de diferentes fontes, como bancos de dados, arquivos, APIs, sistemas transacionais ou fluxos em tempo real. Em um ambiente de produção, essa camada deve ser desacoplada da lógica de Machine Learning, permitindo que a origem dos dados seja alterada sem impactar as etapas seguintes. Uma boa arquitetura trata a ingestão como um componente substituível, responsável apenas por fornecer dados no formato esperado pelo sistema.

Após a ingestão, os dados passam por transformações necessárias para que possam ser utilizados pelo modelo. Isso inclui limpeza, tratamento de valores ausentes, padronização, codificação de variáveis categóricas e criação de atributos derivados. Em sistemas bem arquitetados, o pré-processamento é tratado como parte da lógica de negócio do ML, garantindo que as mesmas regras sejam aplicadas tanto no treinamento quanto na predição, evitando inconsistências entre ambientes.

Já treinamento é a etapa em que o modelo aprende padrões a partir dos dados processados. Aqui são definidos o algoritmo, os hiperparâmetros e o conjunto de dados utilizado. Em arquiteturas maduras, o treinamento não depende diretamente de detalhes de infraestrutura, como onde os dados estão armazenados ou como o modelo será servido depois. Essa separação permite trocar algoritmos, ajustar estratégias ou até mudar bibliotecas sem reescrever o sistema inteiro.

A validação, por sua vez, tem como objetivo avaliar se o modelo treinado atende aos critérios de qualidade definidos, como desempenho estatístico, estabilidade e generalização. Essa etapa funciona como um ponto de controle antes que o modelo seja utilizado em produção. Uma arquitetura bem estruturada trata a validação como uma decisão de negócio, permitindo bloquear modelos inadequados e registrar métricas para auditoria e rastreabilidade.

Após aprovado, o modelo precisa ser armazenado de forma segura e versionada. A persistência envolve salvar o artefato treinado, seus metadados, parâmetros e métricas associadas. Essa etapa é essencial para garantir reprodutibilidade, rollback e auditoria. Em termos arquiteturais, a forma de persistência deve ser desacoplada do núcleo do sistema, permitindo alterar o mecanismo de armazenamento sem afetar o restante do pipeline.

O serviço de predição é a interface entre o modelo e o mundo externo. Ele recebe dados de entrada, aplica o mesmo pré-processamento utilizado no treinamento e retorna previsões para outros sistemas ou usuários. Essa camada precisa ser confiável, escalável e facilmente monitorável. Em arquiteturas modernas, o serviço de predição é tratado como um componente independente, o que facilita sua evolução, escalabilidade horizontal e integração com diferentes consumidores.

Monólitos vs. microsserviços em aplicações de machine learning

Ao desenvolver aplicações de Machine Learning, não lidamos apenas com modelos estatísticos. Também precisamos projetar sistemas de software que sejam capazes de coletar dados, treinar modelos, realizar inferências e evoluir ao longo do tempo. A forma como esses componentes são organizados é definida pela arquitetura de serviços adotada.

Duas abordagens arquiteturais muito comuns são a arquitetura monolítica e a arquitetura de microsserviços. Cada uma possui vantagens e limitações, especialmente quando aplicadas a sistemas de ML.

Agora, começaremos pela arquitetura monolítica, que costuma ser o ponto de partida natural em muitos projetos.

Arquitetura monolítica

Uma arquitetura monolítica é aquela em que todo o sistema é construído e implantado como uma única aplicação. Isso significa que diferentes responsabilidades — como leitura de dados, processamento, treinamento de modelos e disponibilização de uma API — estão todas concentradas no mesmo código-base e são executadas dentro do mesmo processo.

Em outras palavras, o sistema funciona como um “bloco único”, no qual todas as partes estão fortemente integradas. Imagine um sistema de ML que lê dados diretamente de um banco ou arquivo; realiza pré-processamento; treina um modelo; salva esse modelo; e disponibiliza uma API para fazer previsões.

Tudo isso acontece dentro da mesma aplicação, como por exemplo um único projeto em Python; um único repositório; um único deploy. Se essa aplicação falhar ou precisar ser atualizada, todo o sistema é afetado, independentemente de qual parte foi modificada.

Apesar das limitações, a arquitetura monolítica possui vantagens importantes, especialmente em estágios iniciais de um projeto.

- **Simplicidade inicial:** é mais fácil de entender e implementar; não exige conhecimento avançado de arquitetura distribuída; ideal para projetos pequenos, provas de conceito (POCs) ou ambientes acadêmicos.
- **Menor custo operacional:** apenas um serviço para manter; menos infraestrutura envolvida; monitoramento e deploy mais simples no início.

Essas características fazem com que a arquitetura monolítica seja muito utilizada em primeiras versões de sistemas de ML.

À medida que o sistema cresce, especialmente em projetos de Machine Learning, surgem limitações importantes, como acoplamento entre treino, inferência e dados. Em ML, treino e inferência têm naturezas muito diferentes: treinamento é pesado, demorado e consome muitos recursos; inferência precisa ser rápida, estável e altamente disponível.

No monólito, essas etapas ficam fortemente acopladas; mudanças no treinamento podem impactar a API de predição; erros em uma parte afetam todo o sistema; e há dificuldade de escalar partes específicas.

Suponha que a API de inferência recebe milhares de requisições por minuto, mas o treinamento ocorre apenas uma vez por dia. Em um monólito não é possível escalar apenas a inferência e o sistema inteiro precisa ser replicado, resultando em desperdício de recursos computacionais; além disso, o deploy também é arriscado. Qualquer alteração — mesmo pequena — exige rebuild completo da aplicação e novo deploy do sistema inteiro. Isso é especialmente perigoso em ML, em que ajustes em features, atualizações de modelos ou correções de bugs podem introduzir falhas inesperadas.

À medida que sistemas de Machine Learning crescem em complexidade e uso, a arquitetura monolítica passa a apresentar limitações importantes. Para lidar com esses desafios, surge a arquitetura de microsserviços, uma abordagem que busca desacoplar responsabilidades e tornar o sistema mais flexível, escalável e evolutivo.

Arquitetura de Microsserviços

A arquitetura de microsserviços organiza o sistema como um conjunto de serviços pequenos, independentes e especializados, onde cada serviço possui uma responsabilidade bem definida, pode ser desenvolvido e implantado de forma independente e comunica-se com outros serviços por meio de APIs ou mensageria. Em vez de um único bloco, o sistema passa a ser formado por múltiplos componentes autônomos, que juntos compõem a aplicação de Machine Learning.

Em um sistema de ML baseado em microsserviços, é comum separar responsabilidades como:

- Serviço de Ingestão de Dados:
 - coleta dados de diferentes fontes (bancos, APIs, arquivos),
 - realiza validações básicas.
- Serviço de Treinamento:
 - executa pipelines de pré-processamento,
 - treina e avalia modelos,
 - versiona modelos e métricas.
- Serviço de Inferência:
 - carrega modelos já treinados,
 - expõe uma API para predição em tempo real,
 - é altamente otimizado para baixa latência.
- Serviço de Monitoramento:
 - acompanha desempenho do modelo,

- detecta drift de dados ou conceito,
- coleta logs e métricas.

Cada um desses serviços pode usar tecnologias diferentes, escalar de forma independente ou ser atualizado sem impactar os demais.

Com relação às vantagens dos microsserviços em aplicações de ML, temos o desacoplamento entre treino, inferência e dados.

Em Machine Learning, diferentes etapas do ciclo de vida do modelo possuem ritmos e requisitos distintos. Microsserviços permitem treinar modelos sem interromper a API de predição, testar novas versões de modelos isoladamente e reduzir impactos colaterais entre componentes. Esse desacoplamento é essencial para sistemas de ML em produção, pois está relacionado à escalabilidade independente.

Com microsserviços, é possível escalar apenas o serviço de inferência em momentos de pico, manter o serviço de treinamento com recursos dedicados e otimizar custos computacionais. Isso resolve um dos principais problemas da arquitetura monolítica em ML.

Apesar das vantagens, a arquitetura de microsserviços não é trivial e traz novos desafios. Entre eles: maior complexidade arquitetural; mais serviços para desenvolver e manter; necessidade de orquestração e comunicação entre serviços; maior esforço de observabilidade (logs, métricas, tracing); custo operacional maior; infraestrutura mais complexa; uso de containers, orquestradores e pipelines; necessidade de monitoramento contínuo.

A arquitetura de microsserviços é mais indicada quando:

- O sistema de ML está em produção e cresce continuamente,
- Há necessidade de escalar inferência independentemente,
- Múltiplos times trabalham em partes diferentes do sistema,
- Modelos precisam ser atualizados com frequência e segurança.

Em contrapartida, para projetos iniciais ou acadêmicos, uma arquitetura monolítica bem organizada pode ser suficiente.

Comparação entre arquitetura monolítica e microsserviços em aplicações de machine learning

Após compreender os conceitos de arquitetura monolítica e arquitetura de microsserviços, é fundamental compará-las diretamente para entender quando e por que cada abordagem é mais adequada, especialmente no contexto de aplicações de Machine Learning.

Sistemas de ML possuem características próprias, como ciclos contínuos de treinamento, necessidade de inferência em tempo real, grande volume de dados e evolução constante dos modelos. Essas características impactam diretamente a escolha da arquitetura.

ASPECTO	ARQUITETURA MONOLÍTICA	ARQUITETURA DE MICROSERVIÇOS
Estrutura	Aplicação única	Conjunto de serviços independentes
Deploy	Único, para todo o sistema	Independente por serviço
Escalabilidade	Global	Fina e específica
Acoplamento	Alto	Baixo
Complexidade inicial	Baixa	Alta
Evolução do sistema	Difícil a longo prazo	Facilitada

Tabela 1 – Visão Geral da Comparação
Fonte: Elaborado pelo autor (2026)

Clean architecture/arquitetura hexagonal aplicada a machine learning

Em projetos de Machine Learning em produção, o principal desafio raramente está no modelo em si, mas na forma como o sistema é estruturado ao seu redor. Problemas comuns surgem quando o código fica fortemente acoplado a bancos de dados ou APIs, quando alterar o modelo exige modificar diversas partes do sistema, quando a realização de testes se torna difícil e quando o projeto deixa de escalar à medida que a equipe cresce. A Clean Architecture, juntamente com sua variação conhecida como Arquitetura Hexagonal, surge como uma abordagem para organizar responsabilidades e proteger o núcleo do sistema, garantindo que o projeto permaneça sustentável, testável e evolutivo ao longo do tempo.

O princípio central da Clean Architecture é organizar o sistema de forma que as regras de negócio permaneçam no centro, enquanto os detalhes técnicos fiquem nas camadas externas. Isso significa que as dependências sempre apontam para o núcleo do sistema, nunca o contrário. No contexto de Machine Learning, isso implica que o modelo, a lógica de treinamento e as regras de validação não devem depender diretamente de tecnologias específicas como bancos de dados, bibliotecas ou APIs externas. Essa separação permite que o sistema evolua sem comprometer suas regras fundamentais, tornando o código mais estável e fácil de manter.

Dentro dessa arquitetura, o domínio representa o coração do sistema e contém as regras fundamentais que definem o comportamento do Machine Learning. Essas regras incluem a definição do tipo de problema, os critérios de avaliação do modelo, as métricas aceitáveis e as decisões que determinam se um modelo pode ser utilizado em produção. Essas regras são independentes de tecnologia e mudam apenas quando o negócio ou os objetivos analíticos mudam. Em contraste, a infraestrutura representa os detalhes técnicos do sistema, como bancos de dados, frameworks de Machine Learning, APIs externas e serviços em nuvem. Esses elementos são necessários para o funcionamento do sistema, mas devem permanecer isolados das regras centrais, pois são mais suscetíveis a mudanças ao longo do tempo.

A regra de dependência é um dos pilares mais importantes dessa arquitetura. Ela estabelece que a infraestrutura depende do domínio, mas o domínio nunca depende da infraestrutura. O domínio define contratos e comportamentos, enquanto as camadas externas implementam esses contratos. Isso garante que o núcleo do

sistema permaneça estável e independente de tecnologias específicas, permitindo que componentes técnicos sejam substituídos sem impactar a lógica principal do sistema.

Complementando essa abordagem, o Domain-Driven Design, ou DDD, é uma metodologia que organiza o sistema com base nos conceitos do domínio do problema, em vez de organizá-lo com base em tecnologias. Em projetos de Machine Learning, isso significa que o código deve refletir conceitos como experimentos, modelos, métricas e validações, em vez de estruturas técnicas genéricas. Essa abordagem facilita a comunicação entre cientistas de dados, engenheiros e stakeholders, pois o código passa a utilizar a mesma linguagem que é usada para descrever o negócio e o processo analítico.

Um conceito fundamental do DDD é a linguagem ubíqua, que representa o uso consistente dos mesmos termos tanto no código quanto nas discussões sobre o sistema. Quando conceitos como experimento, modelo e métrica são representados explicitamente no código, o sistema se torna mais fácil de entender e manter, reduzindo ambiguidades e aumentando a clareza do projeto.

Dentro do DDD, dois conceitos centrais são as entidades e os objetos de valor. Uma entidade é um objeto que possui identidade própria e ciclo de vida independente. Em Machine Learning, um experimento é um exemplo claro de entidade, pois possui um identificador único, um estado e métricas associadas e continua sendo o mesmo experimento mesmo quando seus atributos mudam. Além de armazenar dados, uma entidade também contém comportamentos, como atualizar métricas, mudar de estado e validar se está apta para produção.

Por outro lado, os objetos de valor representam conceitos definidos exclusivamente por seus atributos e não possuem identidade própria. Em Machine Learning, métricas e hiperparâmetros são exemplos típicos de objetos de valor, pois dois conjuntos com os mesmos valores são considerados equivalentes. Esses objetos devem ser imutáveis, o que garante consistência e facilita testes e validações. A distinção entre entidades e objetos de valor torna o modelo do domínio mais claro, organizado e alinhado com os conceitos reais do sistema.

Outro conceito importante é o agregado, que representa um conjunto de entidades e objetos de valor tratados como uma única unidade de consistência. O

agregado garante que as regras do domínio sejam respeitadas e que mudanças ocorram de forma controlada. Cada agregado possui uma raiz, chamada de Aggregate Root, que é a única entidade acessível diretamente por outras partes do sistema. No contexto de Machine Learning, o experimento pode atuar como raiz do agregado, sendo responsável por controlar métricas, hiperparâmetros e mudanças de estado, garantindo que todas as alterações sejam consistentes com as regras definidas.

Além disso, o padrão Unit of Work é utilizado para coordenar operações relacionadas e garantir consistência no sistema. Ele funciona como um gerenciador de transações, assegurando que um conjunto de operações seja tratado como uma única unidade lógica. Isso significa que todas as operações devem ser concluídas com sucesso para que as mudanças sejam confirmadas, ou, em caso de falha, todas as alterações são revertidas. Em pipelines de Machine Learning, esse conceito é essencial, pois processos como treinamento, cálculo de métricas e persistência do modelo fazem parte de um único fluxo lógico. Se uma etapa falhar, as demais não devem ser concluídas, evitando estados inconsistentes, como métricas salvas sem um modelo válido ou modelos persistidos sem validação adequada.

Ao aplicar Clean Architecture e Domain-Driven Design em projetos de Machine Learning, o sistema se torna mais organizado, modular e resiliente a mudanças. O domínio permanece protegido de detalhes técnicos, permitindo que o sistema evolua sem comprometer sua integridade. Essa abordagem facilita testes, melhora a colaboração entre equipes e garante que o código reflita corretamente as regras do negócio e os conceitos estatísticos envolvidos, resultando em sistemas de Machine Learning mais robustos, confiáveis e sustentáveis ao longo do tempo.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, o foco foi ampliar a visão sobre Machine Learning, indo além do desenvolvimento de modelos isolados e explorando como soluções de ML são pensadas, estruturadas e mantidas em sistemas reais. O objetivo central foi mostrar que Machine Learning, quando aplicado em contextos profissionais, é tão dependente de boas decisões arquiteturais quanto de bons algoritmos.

Um dos temas centrais foi a comparação entre arquiteturas monolíticas e microsserviços, especialmente no contexto de aplicações de Machine Learning. Foi discutido que não existe uma solução universalmente melhor, mas sim escolhas arquiteturais que dependem do contexto, do tamanho do sistema e da maturidade da equipe. Monólitos podem ser simples, eficientes e adequados para muitos cenários iniciais, enquanto microsserviços podem trazer benefícios em escalabilidade e independência, mas também aumentam a complexidade operacional.

Em seguida, a aula avançou para padrões arquiteturais modernos, com destaque para a Clean Architecture e a Arquitetura Hexagonal. Esses padrões foram apresentados como formas de organizar sistemas colocando o domínio no centro, protegendo as regras de negócio e a lógica de Machine Learning de detalhes técnicos.

Outro conceito fundamental apresentado foi o Domain-Driven Design (DDD) em uma visão introdutória. O DDD foi apresentado como uma forma de modelar o sistema a partir dos conceitos do negócio, utilizando uma linguagem comum entre desenvolvedores(as), cientistas de dados e stakeholders. A ideia de linguagem ubíqua mostrou como nomear corretamente classes, métodos e estruturas ajuda a reduzir ambiguidades e alinhar o código com o problema real que está sendo resolvido.

Dentro do DDD, foram explorados os conceitos de Entidades e Objetos de Valor, com exemplos aplicados a Machine Learning. Entidades foram apresentadas como objetos que possuem identidade própria e ciclo de vida, como um experimento de ML específico, que pode mudar de estado ao longo do tempo. Objetos de Valor, por sua vez, foram definidos como estruturas imutáveis, sem identidade, representando conceitos como métricas e hiperparâmetros. Essa distinção ajuda a organizar melhor o domínio e a tornar as regras do sistema mais explícitas e seguras.

Por fim, foi introduzido o conceito de Unit of Work, apresentado como um padrão avançado e de caráter conceitual. O Unit of Work foi discutido como uma forma de coordenar múltiplas operações relacionadas, garantindo consistência por meio de commits e rollbacks. No contexto de Machine Learning, isso foi conectado a processos como treino de modelos, cálculo de métricas e persistência de artefatos, reforçando a importância de tratar essas etapas como uma única unidade lógica. Foi destacado que esse padrão não deve ser implementado neste momento, mas compreendido como parte do vocabulário arquitetural de sistemas maduros de ML.

Ao longo da aula, o ponto principal foi reforçado diversas vezes: Machine Learning não é apenas sobre modelos, mas sobre como esses modelos são integrados em sistemas reais. Arquitetura, organização de código e padrões de projeto são fundamentais para garantir que soluções de ML sejam confiáveis, testáveis, escaláveis e sustentáveis ao longo do tempo. A aula buscou preparar o aluno e a aluna para enxergar o ML como parte de um ecossistema maior, aproximando a prática acadêmica da realidade do mercado e de projetos profissionais.

REFERÊNCIAS

COCKBURN, A. **Hexagonal architecture (ports and adapters)**. [S. l.]: [s. n.], 2005.

EVANS, E. **Domain-Driven Design: tackling complexity in the heart of software**. Boston: Addison-Wesley, 2003.

FOWLER, M. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GOOGLE CLOUD. **Machine learning architecture patterns**. [S. l.]: Google, [s. d.].

MARTIN, R. C. **Clean Architecture: a craftsman's guide to software structure and design**. Boston: Prentice Hall, 2017.

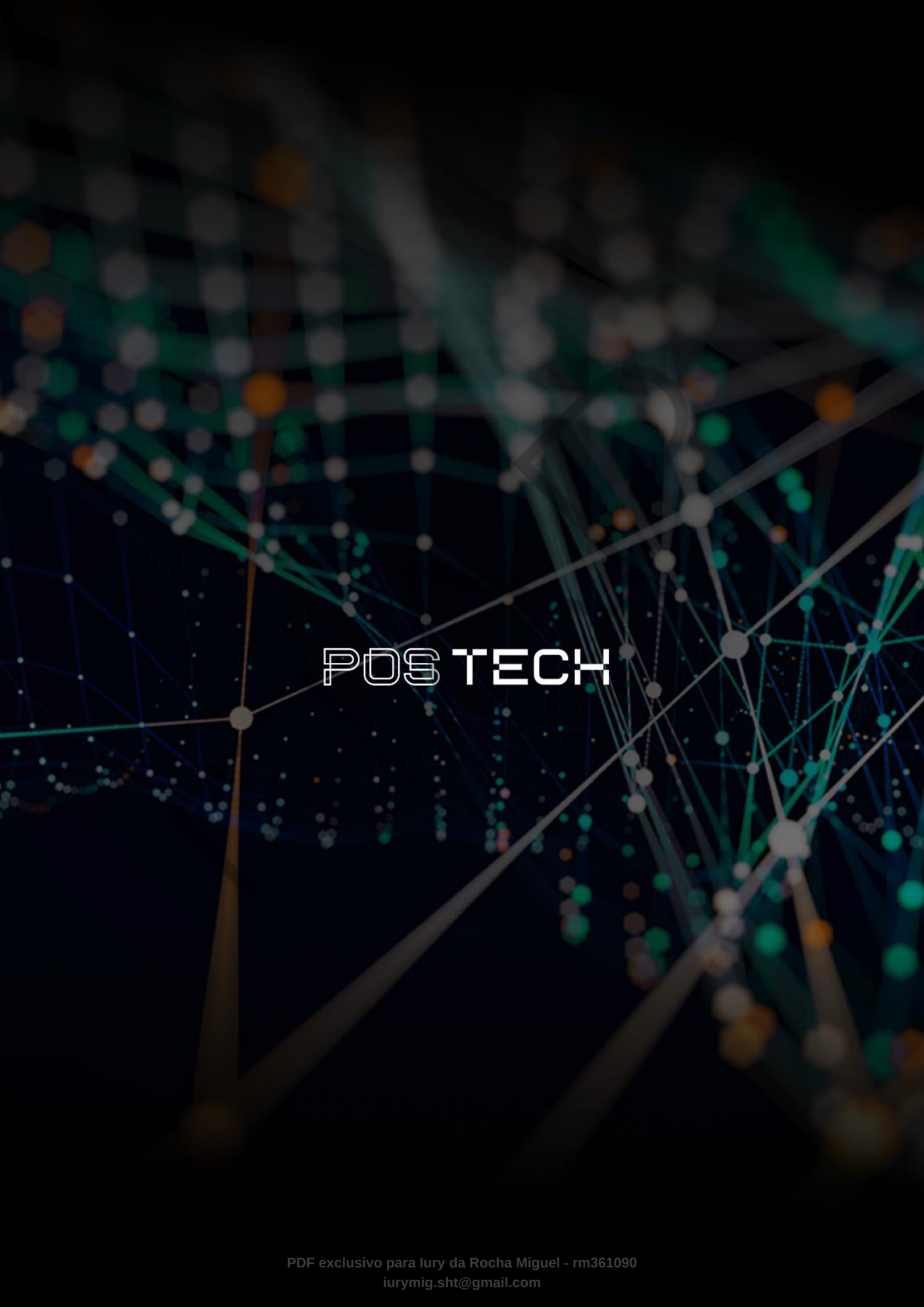
PERCIVAL, H.; GREGORY, B. **Architecture Patterns with Python**. Sebastopol: O'Reilly Media, 2020.

SUFYAN, U. **How to approach any machine learning problem**. 2022. Disponível em: <https://medium.com/divedeeai/how-to-approach-any-machine-learning-problem-34f01995c17>. Acesso em: 16 mar. 2026.

PALAVRAS-CHAVE

Arquitetura. Governança. Produção.

EMSE



POSTECH